6 Epidermal wound healing

The PDE models discussed in this chapter pertain to wound healing. We consider two models ([3]), based on one PDE and two PDEs, respectively. To start with the one-PDE model, the movement of epithelial cells across the wound is modeled by the following nonlinear form of the 1D diffusion equation with an additional nonlinear logistic (algebraic) term.

6.1 One-PDE model

$$\frac{\partial u}{\partial t} = D\frac{\partial\left((1-u/u_0)^p\dfrac{\partial u}{\partial r}\right)}{\partial r} + s_c u(1-u/u_0), \tag{6.1}$$

where

Variable, parameter set:	Interpretation, units
u	density of epithelial cells (cells/cm^3)
r	lateral position across the wound (cm)
t	time (s)
u_0	normal (unwounded) cell density (cells/cm^3)
D	cell diffusivity (cm^2/s)
p	parameter in the function for nonlinear diffusion (dimensionless)
s_c	coefficient of the nonlinear logistic source term (1/s)

A detailed discussion of this model in given in [3].[1] We use r as the spatial independent variable because of this choice in [3]; eq. (6.1) is in Cartesian coordinates, not radial coordinates.

[1] The original variable diffusivity $(u/u_0)^p$ is changed to $(1-u/u_0)^p$ to avoid computational problems such as the indeterminate form 0^0 (with $u = p = 0$) and a zero RHS of eq. (6.1) when $u = 0$ so that $(\partial u/\partial t) = 0$ and therefore $u(r,t)$ does not change with t. The computational approach to the variable diffusivity is the principal focus here, and is illustrated with the alternate form of the diffusivity. This alternate form $((1-u/u_0)^p)$ can also take the indeterminate form 0^0 with $u = u_0, p = 0$, but this is less likely ($u = u_0$ is less likely than $u = 0$ in the model and the numerical analysis that follows).

The solution of eq. (6.1) is the cell density $u(r,t)$, which we take as a normalized variable with $u_0 = 1$. The model can easily be restated in terms of a cell density with an absolute value (and units).

Equation (6.1) is first order in t and second order in r so that it requires one IC and two BCs.

$$u(r, t = 0) = 0, \tag{6.2}$$

$$u(r = 0, t) = u_0 = 1, \ \frac{\partial u(r = r_0, t)}{\partial r} = 0, \tag{6.3a,b}$$

where $r = r_0$ is the lateral half width of the wound (cm). Initial condition (6.2) states that the wound initially has no epithelial cells. Boundary condition (6.3a) specifies that the density of epithelial cells at the left end (edge) of the wound is a normal (unit) value and BC (6.3b) specifies a zero-flux condition, which can be interpreted as a symmetry condition at the midpoint of the wound, $r = r_0$ (an IC and BCs for eq. (6.1) were not given in [3], so these auxiliary conditions are inferred from the plotted solutions in this reference). We next develop a MOL solution to eqs. (6.1) to (6.3) that expresses the diffusion and regeneration of epithelial cells during the healing process.

6.1.1 Main program

A main program for the solution of eqs. (6.1) to (6.3) is given in Listing 6.1.

```
%
% Wound healing
%
% Clear previous files
  clear all
  clc
  global     nr     r0      r     u0...
              D      p     sc  ncall
%
% Model parameters
  r0=0.5; u0=1; D=2.0e-09; sc=      0; p=0;
% r0=0.5; u0=1; D=2.0e-09; sc=8.0e-06; p=0;
% r0=0.5; u0=1; D=2.0e-09; sc=8.0e-06; p=1;
% r0=0.5; u0=1; D=2.0e-09; sc=8.0e-06; p=5;
%
% Spatial grid
  nr=101;
  dr=r0/(nr-1);
  for j=1:nr
    r(j)=(j-1)*dr;
  end
%
% Independent variable for ODE integration
  tf=60*60*24*30;
  tout=[0:tf/10:tf]';
  nout=11;
  ncall=0;
```

```
%
% Initial condition
  for i=1:nr u0w(i)=0; end
%
% ODE integration
  reltol=1.0e-04; abstol=1.0e-04;
  options=odeset('RelTol',reltol,'AbsTol',abstol);
  [t,u]=ode15s(@pde_1,tout,u0w,options);
%
% Display a heading and t at output
  fprintf('\n  nr = %2d\n',nr);
  for it=1:nout
    u(it,1)=u0;
    fprintf('\n t (s) = %6.2e   t (days) = %6.2f',...
            t(it),t(it)/(60*60*24));
  end
  fprintf('\n\n ncall = %5d\n',ncall);
%
% Plot the solution as u(r,t) vs r with t as a parameter
  plot(r,u(2:nout,:),'-');
  axis([0 0.5 0 1])
  xlabel('r (cm)'); ylabel('u(r,t)');
  title('u(r,t);t = 3, 6,..., 30 days')
```

Listing 6.1 Main program `pde_1_main` for the MOL solution of eqs. (6.1) to (6.3)

We can note the following points about `pde_1_main`.

- Previous files are cleared and a global area is defined.

```
%
% Wound healing
%
% Clear previous files
  clear all
  clc
  global     nr     r0      r     u0...
              D      p     sc  ncall
```

  Also, documentation comments are not included for the model equations since they change when the single-PDE model of this section is extended to a two-PDE model in a subsequent section.
- Four sets of model parameters are programmed, and are deactivated through the use of a comment (with beginning %). Here the first set (line) of comments is active.

```
%
% Model parameters
  r0=0.5; u0=1; D=2.0e-09; sc=      0; p=0;
% r0=0.5; u0=1; D=2.0e-09; sc=8.0e-06; p=0;
% r0=0.5; u0=1; D=2.0e-09; sc=8.0e-06; p=1;
% r0=0.5; u0=1; D=2.0e-09; sc=8.0e-06; p=5;
```

An alternative for the selection of the parameters would be to use an `if` or `switch case` construct (but using the comment is just as easy).

For the first line of parameters, eq. (6.1) reduces to the 1D diffusion equation (1.18). Then eqs. (6.1) to (6.3) have a well-known analytical (infinite series) solution, but we will not use it here for the evaluation of the numerical solution since the PDE problem of eqs. (6.1) to (6.3) is a straightforward MOL application (the reader may wish to add an extension with the analytical solution).

The value of the diffusivity `D` is small (2×10^{-9} cm/s) but is not unusual for a biological system, and accounts for a time scale of one month (discussed subsequently).

- A spatial grid in r of `101` points is defined with grid spacing `dr` over the interval $0 \le r \le r_0$.

```
%
% Spatial grid
  nr=101;
  dr=r0/(nr-1);
  for j=1:nr
    r(j)=(j-1)*dr;
  end
```

- The interval in t is $0 \le t \le (60)(60)(24)(30)$, which is one month in seconds (s). The time scale in t is in s because the diffusivity D in eq. (6.1) has the units cm^2/s.

```
%
% Independent variable for ODE integration
  tf=60*60*24*30;
  tout=[0:tf/10:tf]';
  nout=11;
  ncall=0;
```

The output interval is $1/10$ of the total interval (`tf/10`) for a total of `11` output points (including $t = 0$). The counter for the number of calls to the ODE routine is initialized to zero.

- The IC of eq. (6.2) is set.

```
%
% Initial condition
  for i=1:nr u0w(i)=0; end
```

- The integration of the `101` ODEs is by `ode15s`.

```
%
% ODE integration
  reltol=1.0e-04; abstol=1.0e-04;

  options=odeset('RelTol',reltol,'AbsTol',abstol);
  [t,u]=ode15s(@pde_1,tout,u0w,options);
```

The sparse version of `ode15s` is not used since the full-matrix version executes quickly.

- A heading is displayed with the number of grid points `nr`. Then the running values of t are displayed to indicate the progress in the solution (the solution `u` could also

be displayed, but this would give 101 numbers for each value of t; this probably too many numbers to easily assimilate but could be visualized in a single plot, programmed next).

```
%
% Display a heading and t at output
  fprintf('\n  nr = %2d\n',nr);
  for it=1:nout
    u(it,1)=u0;
    fprintf('\n t (s) = %6.2e   t (days) = %6.2f',...
            t(it),t(it)/(60*60*24));
  end
  fprintf('\n\n ncall = %5d\n',ncall);
```

The running values of t are displayed in both seconds and days. Note also that the boundary value specified by BC (6.3a) is stored in `u(it,1)`, since this value is not computed by an ODE and therefore is not returned from the ODE routine `pde_1` via `ode15s`. The final value of the counter for the calls to `pde_1` is also displayed to give an indication of the total effort required to compute the complete numerical solution.

- The solution is plotted as $u(r,t)$ against r with t as a parameter.

```
%
% Plot the solution as u(r,t) vs r with t as a parameter
  plot(r,u(2:nout,:),'-');
  axis([0 0.5 0 1])
  xlabel('r (cm)'); ylabel('u(r,t)');
  title('u(r,t);t = 3, 6,..., 30 days')
```

The starting value of t is 3 days, since the solution at $t = 0$ undergoes a discontinuity at $r = 0$, that is, $u(r = 0, t = 0) = 0$ to $u(r = 0, t > 0) = u_0$, that is not well suited for plotting. However, the smoothness of the plotted solution for $t > 0$ in Fig. 6.1 (discussed subsequently) illustrates an important feature of parabolic PDEs such as eq. (6.1), that is, they tend to smooth discontinuities (which is not the case for hyperbolic PDEs, as demonstrated in Section 1.2.4); this property is suggested by the name of eq. (6.1), the diffusion equation, since it tends to diffuse (or smooth) discontinuities.

6.1.2 ODE routine

The ODE routine `pde_1` called by `ode15s` in `pde_1_main` is listed next.

```
  function ut=pde_1(t,u)
%
% Global area
  global     nr     r0      r     u0...
              D      p     sc  ncall
%
% BC at r = 0
  u(1)=u0; nl=1;
```

```
%
% ur
  ur=dss004(0,r0,nr,u);
%
% BC ar r = r0
  ur(nr)=0; nu=2;
%
% urr
  urr=dss044(0,r0,nr,u,ur,nl,nu);
%
% ut
  for i=2:nr
    ut(i)=D*((1-u(i)/u0)^p*urr(i)+p*(1-u(i)/u0)^(p-1)*(-1/u0)*ur(i)^2)+...
          sc*u(i)*(1-u(i)/u0);
  end
  ut(1)=0;
%
% Transpose and counter
  ut=ut';
  ncall=ncall+1;
```

Listing 6.2 ODE routine `pde_1` for eqs. (6.1) to (6.4)

We can note the following details of `pde_1`.

- The function and a global area are defined.

```
  function ut=pde_1(t,u)
%
% Global area
  global     nr      r0       r      u0...
              D       p      sc  ncall
```

- Boundary condition (6.3a) is programmed as a Dirichlet BC (with `nl=1`). Note that this BC resets `u` at $r = 0$, which comes into `pde_1` as a (RHS) argument.

```
%
% BC at r = 0
  u(1)=u0; nl=1;
```

- The derivative $(\partial u/\partial r)$ is computed by a call to `dss004`.

```
%
% ur
  ur=dss004(0,r0,nr,u);
```

- Boundary condition (6.3b) is programmed as a Neumann BC (with `nu=2`), which resets the derivative $(\partial u/\partial r)$ from `dss004` at the right boundary $(r = r_0)$.

```
%
% BC ar r = r0
  ur(nr)=0; nu=2;
```

- The second derivative $(\partial^2 u/\partial r^2)$ is computed by a call to `dss044`.

```
%
% urr
  urr=dss044(0,r0,nr,u,ur,nl,nu);
```

- The PDE, eq. (6.1), is programmed. First, the term in r in eq. (6.1) is expanded as the derivative of a product.

$$\frac{\partial\left((1-u/u_0)^p\dfrac{\partial u}{\partial r}\right)}{\partial r} = (1-u/u_0)^p\frac{\partial^2 u}{\partial r^2} + \frac{\mathrm{d}\left((1-u/u_0)^p\right)}{\mathrm{d}u}\frac{\partial u}{\partial r}\frac{\partial u}{\partial r}$$
$$= (1-u/u_0)^p\frac{\partial^2 u}{\partial r^2} + \left(p(1-u/u_0)^{p-1}(-1/u_0)\right)\left(\frac{\partial u}{\partial r}\right)^2. \tag{6.4}$$

  The programming of eq. (6.1), with the substitution of eq. (6.4), follows directly.

```
%
% ut
  for i=2:nr
    ut(i)=D*((1-u(i)/u0)^p*urr(i)+p*(1-u(i)/u0)^(p-1)*(-1/u0)
          *ur(i)^2)+... sc*u(i)*(1-u(i)/u0);
  end
  ut(1)=0;
```

  The left end derivative in t, $(\partial u(r=0,t)/\partial t)$, is set to zero so that the boundary value remains at $u_0 = 1$, as specified by eq. (6.3a).

- The transpose required by `ode15s` and incrementing of the counter for the calls to `pde_1` concludes `pde_1`.

```
%
% Transpose and counter
  ut=ut';
  ncall=ncall+1;
```

  This completes the programming of eqs. (6.1) to (6.4).

6.1.3 Model output

We now consider the output for successive lines of the programming of the parameters in `pde_1`, starting with the linear case $p = s = 0$.

Linear diffusion equation

The numerical output is given in Table 6.1 (again, for the first line of the parameters in `pde_1_main`), and the graphical output is in Fig. 6.1. Only the running value of t appears in the numerical output of Table 6.1, to conserve space.

The solution is computed with modest effort (`ncall=196`).

The plotted solution in Fig. 6.1 is for the diffusion equation (eq. (6.1) with $p = s = 0$) with a constant Dirichlet BC at $r = 0$ and a homogeneous Neumann BC at $r = r_0$. It is offered as a point of reference for subsequent solutions that have a much different form, due to changes in p and s.

Table 6.1. Output from `pde_1_main` and `pde_1` for `r0=0.5; u0=1; D=2.0e-09; sc=0; p=0;`

```
nr = 101

t (s) = 0.00e+000   t (days) =   0.00
t (s) = 2.59e+005   t (days) =   3.00
t (s) = 5.18e+005   t (days) =   6.00
t (s) = 7.78e+005   t (days) =   9.00
t (s) = 1.04e+006   t (days) =  12.00
t (s) = 1.30e+006   t (days) =  15.00
t (s) = 1.56e+006   t (days) =  18.00
t (s) = 1.81e+006   t (days) =  21.00
t (s) = 2.07e+006   t (days) =  24.00
t (s) = 2.33e+006   t (days) =  27.00
t (s) = 2.59e+006   t (days) =  30.00

ncall =   196
```

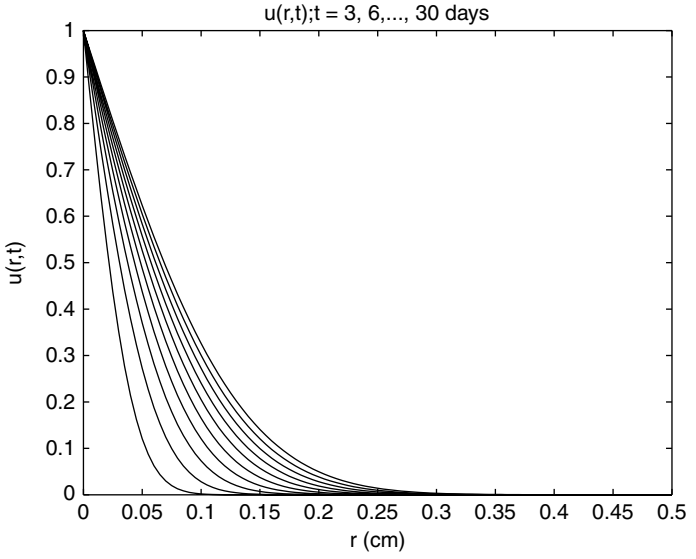


Figure 6.1 Solution to eqs. (6.1) to (6.4) for $p = s_c = 0$

This solution has the limiting value $u(r, t \to \infty) = 1$ since the left boundary defines a constant value $u_0 = 1$ and the right boundary has a no-flux condition (mass cannot leave the system). However, the final value of $t = 30$ days is not long enough to approach this limiting solution, and the time scale in `pde_1_main` would have to be increased substantially to approach the limiting solution. We do not do that here, since a final value $t = 30$ days is more appropriate for the solutions that follow.

Table 6.2. Output from `pde_1_main` and `pde_1` for `r0=0.5; u0=1; D=2.0e-09; sc=8.0 e-06; p=0;`

```
nr = 101

t (s) = 0.00e+000   t (days) =   0.00
t (s) = 2.59e+005   t (days) =   3.00
t (s) = 5.18e+005   t (days) =   6.00
t (s) = 7.78e+005   t (days) =   9.00
t (s) = 1.04e+006   t (days) =  12.00
t (s) = 1.30e+006   t (days) =  15.00
t (s) = 1.56e+006   t (days) =  18.00
t (s) = 1.81e+006   t (days) =  21.00
t (s) = 2.07e+006   t (days) =  24.00
t (s) = 2.33e+006   t (days) =  27.00
t (s) = 2.59e+006   t (days) =  30.00

ncall =   319
```

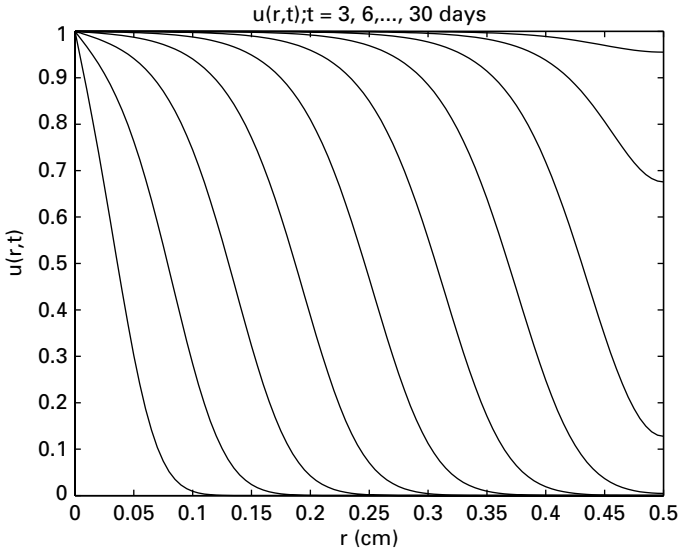


Figure 6.2 Solution to eqs. (6.1) to (6.4) for $p = 0, s_c = 8 \times 10^{-6}$

Fisher–Kolmogorov equation

For the second line of the parameter values in `pde_1_main`,

```
r0=0.5; u0=1; D=2.0e-09; sc=8.0e-06; p=0;
```

the nonlinear logistic source term of eq. (6.1), $s_c u(1 - u/u_0)$, is introduced (owing to the nonzero value of s_c). The resulting PDE is the Fisher–Kolmogorov equation ([2].

Table 6.3. Output from `pde_1_main` and `pde_1` for `r0=0.5; u0=1; D=2.0e-09; sc=8.0e-06; p=1;`

```
nr = 101

t (s) = 0.00e+000   t (days) =   0.00
t (s) = 2.59e+005   t (days) =   3.00
t (s) = 5.18e+005   t (days) =   6.00
t (s) = 7.78e+005   t (days) =   9.00
t (s) = 1.04e+006   t (days) =  12.00
t (s) = 1.30e+006   t (days) =  15.00
t (s) = 1.56e+006   t (days) =  18.00
t (s) = 1.81e+006   t (days) =  21.00
t (s) = 2.07e+006   t (days) =  24.00
t (s) = 2.33e+006   t (days) =  27.00
t (s) = 2.59e+006   t (days) =  30.00

ncall =   437
```

Chapter 8). All of the other coding in `pde_1_main` and `pde_1` remains unchanged. The numerical output is in Table 6.2 and the graphical output is in Fig. 6.2.

The solution is computed with modest effort (`ncall=319`).

The plotted solution in Fig. 6.2 is quite different from the solution in Fig. 6.1 because of the inclusion of the logistic source term. Specifically, the solution is now more like a traveling wave with $p = 0, s_c = 8 \times 10^{-6}$. In other words, the logistic source term has a major effect on the form of the solution (even though it does not include a spatial derivative such as $(\partial u/\partial r)$ for convection, which might be expected considering the form of the traveling wave solution in Fig. (6.2)). Clearly, s_c in eq. (6.1) is a sensitive parameter.

Nonlinear diffusion

For the third line of parameter values in `pde_1_main`,

```
r0=0.5; u0=1; D=2.0e-09; sc=8.0e-06; p=1;
```

p in eq. (6.1) is now nonzero and thus makes the diffusion term in eq. (6.1) nonlinear. This change does not produce a substantial variation in the solution of Fig. 6.2. The numerical and graphical output are in Table 6.3 and Fig. 6.3, respectively.

The solution is computed with modest effort (`ncall=437`). The plotted solution in Fig. 6.3 is similar to the solution in Fig. 6.2 so that p appears to not be a sensitive parameter.

To further examine the effect of p, we consider the fourth line of parameter values in `pde_1_main`.

```
r0=0.5; u0=1; D=2.0e-09; sc=8.0e-06; p=5;
```

This change ($p = 1$ to $p = 5$) does not produce a substantial variation in the solutions of Figs. 6.2 and 6.3. The numerical and graphical output are in Table 6.4 and Fig. 6.4, respectively.

Table 6.4. Output from `pde_1_main` and `pde_1` for `r0=0.5; u0=1; D=2.0e-09; sc=8.0e-06; p=5;`

```
nr = 101

t (s) = 0.00e+000   t (days) =   0.00
t (s) = 2.59e+005   t (days) =   3.00
t (s) = 5.18e+005   t (days) =   6.00
t (s) = 7.78e+005   t (days) =   9.00
t (s) = 1.04e+006   t (days) =  12.00
t (s) = 1.30e+006   t (days) =  15.00
t (s) = 1.56e+006   t (days) =  18.00
t (s) = 1.81e+006   t (days) =  21.00
t (s) = 2.07e+006   t (days) =  24.00
t (s) = 2.33e+006   t (days) =  27.00
t (s) = 2.59e+006   t (days) =  30.00

ncall =   558
```

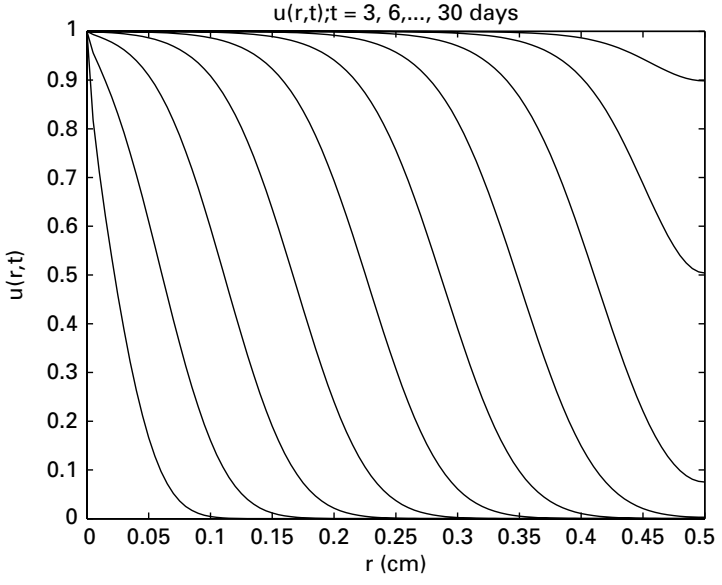


Figure 6.3 Solution to eqs. (6.1) to (6.4) for $p = 1, s_c = 8 \times 10^{-6}$

The solution is computed with modest effort (`ncall=558`). Figure 6.4 has small differences from Figs. 6.2 and 6.3 so that again p appears to not be a sensitive parameter.

To conclude this discussion, we consider an estimation of the apparent velocity of the moving front in Fig. 6.2 for the Fisher–Kolmogorov equation (linear diffusion with $p = 0$ in eq. (6.1)), which gives an indication of the speed of wound healing.

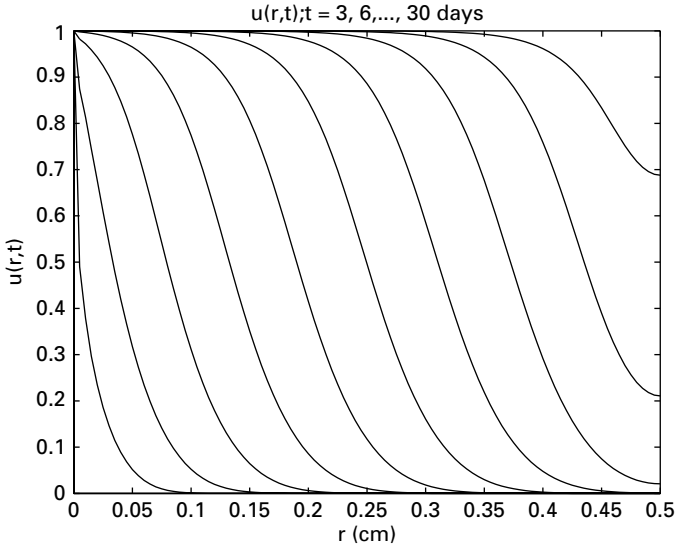


Figure 6.4 Solution to eqs. (6.1) to (6.4) for $p = 5, s_c = 8 \times 10^{-6}$

Velocity of wound healing

To investigate the velocity of wound healing, we add some additional lines of code to `pde_1_main` in Listing 6.1 (just before the plotting at the end).

```
%
% Estimated velocity for p = 0
  for it=4:5
    fprintf('\n\n t = %6.2f',t(it));
    for i=1:nr
      fprintf('\n r = %5.3f   u(r,t) = %5.3f',r(i),u(it,i));
    end
  end
  vs=1.0e+04*(0.175-0.120)/(1036800-777600);
  fprintf('\n\n velocity = %7.5f microns/s\n',vs);
  vday=(0.175-0.120)/(1036800-777600)*3600*24;
  fprintf('\n velocity = %6.4f cm/day\n',vday);
  rmon=(0.175-0.120)/(1036800-777600)*3600*24*30;
  fprintf('\n distance/month = %6.3f cm/mo\n',rmon);
  fprintf('\n wound half width = %6.3f cm\n',r0);
  fprintf('\n ncall = %5d\n',ncall);
```

Listing 6.3 Additional coding added to `pde_1_main` to compute the velocity of the moving front in Fig. 6.2 (p = 0)

We can note the following details about this code.

- Figure 6.2 indicates that the solution to eqs. (6.1) to (6.4) is a moving front that retains an essentially constant form or shape over much of the solution for $0 \leq t \leq 30$. We can estimate the effective velocity of the moving front by observing how far it moves in a

selected interval of time. To do this, we need a more detailed display of the solution. This is done at the fourth and fifth outputs corresponding to $t = 9$ and $t = 12$ days, respectively, in a `for` loop `for it=4:5` (refer to Table 6.2 or Table 6.5 (next) for the corresponding t).

```
%
% Estimated velocity for p = 0
  for it=4:5
    fprintf('\n\n t = %6.2f',t(it));
    for i=1:nr
      fprintf('\n r = %5.3f   u(r,t) = %5.3f',r(i),u(it,i));
    end
  end
```

- From Table 6.5, for `t = 777600.00` s, $u(r,t) = 0.6 \approx 0.594$ is at `r = 0.120`.

```
t = 777600.00
r = 0.120   u(r,t) = 0.594
```

  (a portion of the output in Table 6.5 was deleted for clarity).
- Similarly, from Table 6.5 for `t = 1036800.00` s, $u(r,t) = 0.6 \approx 0.601$ is at `r = 0.175`

```
t = 1036800.00
r = 0.175   u(r,t) = 0.601
```

- Thus the velocity of the front at $u(r,t) \approx 0.6$ is $(0.175 - 0.120/1036800 - 777600)$ cm/s. This calculation is programmed as

```
vs=1.0e+04*(0.175-0.120)/(1036800-777600);
fprintf('\n\n velocity = %7.5f microns/s\n',vs);
```

  where the factor `1.0e+04` is used to convert from cm to μm. Thus, the velocity `vs` is in μm/s.
- Similarly, the velocity in cm/day is computed as

```
vday=(0.175-0.120)/(1036800-777600)*3600*24;
fprintf('\n velocity = %6.4f cm/day\n',vday);
```

- The total distance in cm traveled by the front in a month is given by

```
rmon=(0.175-0.120)/(1036800-777600)*3600*24*30;
fprintf('\n distance/month = %6.3f cm/mo\n',rmon);
```

  which gives an indication of the healing after one month.
- For reference, the wound half width r_0 is displayed along with the number of calls to `pde_1` (as included in previous examples).

```
fprintf('\n wound half width = %6.3f cm\n',r0);
fprintf('\n ncall = %5d\n',ncall);
```

We can now consider the output from the additional code in Listing 6.3 as given in Table 6.5

We can note the following points about the output in Table 6.5.

Table 6.5. Abbreviated output from additional code in Listing 6.3 in `pde_1_main`

```
nr = 101

t (s) = 0.00e+000   t (days) =   0.00
t (s) = 2.59e+005   t (days) =   3.00
t (s) = 5.18e+005   t (days) =   6.00
t (s) = 7.78e+005   t (days) =   9.00
t (s) = 1.04e+006   t (days) =  12.00
t (s) = 1.30e+006   t (days) =  15.00
t (s) = 1.56e+006   t (days) =  18.00
t (s) = 1.81e+006   t (days) =  21.00
t (s) = 2.07e+006   t (days) =  24.00
t (s) = 2.33e+006   t (days) =  27.00
t (s) = 2.59e+006   t (days) =  30.00

t = 777600.00
r = 0.000   u(r,t) = 1.000
r = 0.005   u(r,t) = 0.996
r = 0.010   u(r,t) = 0.992
     .               .
     .               .
     .               .
r = 0.115   u(r,t) = 0.635
r = 0.120   u(r,t) = 0.594
r = 0.125   u(r,t) = 0.550
     .               .
     .               .
     .               .
r = 0.490   u(r,t) = 0.000
r = 0.495   u(r,t) = 0.000
r = 0.500   u(r,t) = 0.000

t = 1036800.00
r = 0.000   u(r,t) = 1.000
r = 0.005   u(r,t) = 0.999
r = 0.010   u(r,t) = 0.998
     .               .
     .               .
     .               .
r = 0.170   u(r,t) = 0.640
r = 0.175   u(r,t) = 0.601
r = 0.180   u(r,t) = 0.560
     .               .
     .               .
     .               .
r = 0.490   u(r,t) = 0.000
r = 0.495   u(r,t) = 0.000
r = 0.500   u(r,t) = 0.000

velocity = 0.00212 microns/s
```

```
velocity = 0.0183 cm/day

distance/month =  0.550 cm/mo

wound half width =  0.500 cm

ncall =   319
```

- The summary of t at the 11 outputs for $0 \le t \le 30$ days is the same as in Fig. 6.1 to 6.4.
- Abbreviated tabular output for `it=4,5` (9 and 12 days) is included, particularly for $u(r,t) \approx 0.6$, as discussed previously.
- The two computed velocities are

```
velocity = 0.00212 microns/s
velocity = 0.0183 cm/day
```

In particular, the second velocity gives an indication of the time in days required for effectively complete wound healing. This point is illustrated for a total healing time of one month.

```
distance/month =  0.550 cm/mo
wound half width =  0.500 cm
```

which indicates that the moving front solution covers the wound half width in approximately one month.

As a point of comparison, an experimental velocity of 8.6×10^{-3} mm/h is reported in [3], p34. From the preceding result (for $p = 0$ in eq. (6.1)), 0.0183 cm/day = $(0.0183)(1/24)(10) = 7.63 \times 10^{-3}$ mm/h. This value is in better agreement with the experimental value than 6×10^{-2} mm/h from [3] for $p = 0$ (linear diffusion). Also, for $p = 4$, the velocity from [3] is 9×10^{-3} mm/h so that there is a significant variation with p (using the original diffusion function $D(u/u_0)^p$), while with the present formulation based on the diffusion function $D(1 - u/u_0)^p$, the variation with p is substantially smaller.

Of course, these results are defined by the specific values of the model parameters, particularly the diffusivity `D=2.0e-09` cm^2/s in the second line of parameter values in `pde_1_main`.

This completes the discussion of the one-PDE model of eqs. (6.1) to (6.4). We now consider a two-PDE model.

6.2 Two-PDE model

The two-PDE model ([1, 3], p35) is an extension of the preceding one-PDE model (eqs. (6.1) to (6.4)) in which the production of epidermal cells through the action of an activator

Table 6.6. Variables, parameters, and terms in the PDEs

Variable, parameter, term	Interpretation
u_1	cell density
u_2	activator or inhibitor concentration
t	time
$\nabla^2 = (\partial^2/\partial r^2) + 1/r(\partial/\partial r)$	laplacian operator (in one dimension)
r	radial direction across the wound
$D_{u_1}\nabla^2 u_1$	net rate of diffusion into or out of a
$D_{u_2}\nabla^2 u_2$	differential volume due to Fickian diffusion for components u_1 and u_2, respectively
$s_c(u_2)u_1(2-u_1)$	volumetric rate of cell mitosis (division)
$-u_1$	volumetric rate of cell depletion
$\lambda g(u_1)$	volumetric rate of chemical (activator or inhibitor) production by the cells
$-\lambda u_2$	volumetric rate of chemical (activator or inhibitor) consumption

and inhibitor is taken into account. The concentration of the activator or inhibitor is the dependent variable for the second PDE. The model PDEs are

$$\frac{\partial u_1}{\partial t} = D_{u_1}\nabla^2 u_1 + s_c(u_2)u_1(2-u_1) - u_1,$$

$$\frac{\partial u_2}{\partial t} = D_{u_2}\nabla^2 u_2 + \lambda g(u_1) - \lambda u_2.$$

The variables, parameters, and terms of these PDEs are listed in Table 6.6. These equations are dimensionless and therefore no dimensions are indicated for the variables, parameters or terms (they are also dimensionless).

If the wound is considered circular, the Laplacian can be expressed in cylindrical coordinates (r,θ,z) in 3D or polar coordinates (r,θ) in 2D. If we consider polar coordinates with no angular dependency, the PDEs become

$$\frac{\partial u_1}{\partial t} = D_{u_1}\left(\frac{\partial^2 u_1}{\partial r^2} + \frac{1}{r}\frac{\partial u_1}{\partial r}\right) + s_c(u_2)u_1(2-u_1) - u_1, \tag{6.5a}$$

$$\frac{\partial u_2}{\partial t} = D_{u_2}\left(\frac{\partial^2 u_2}{\partial r^2} + \frac{1}{r}\frac{\partial u_2}{\partial r}\right) + \lambda g(u_1) - \lambda u_2. \tag{6.5b}$$

The terms described in Table 6.6 indicate that eqs. (6.5) are time-dependent balances on the epidermal cells and the concentration of the activator or inhibitor (two cases). The functions $g(u_1)$ and $s_c(u_2)$ in eqs. (6.5) are different for the activator and inhibitor.

- Activator:

$$g(u_1) = \frac{u_1(1+\alpha^2)}{u_1^2+\alpha^2}, \tag{6.5c}$$

$$s_c(u_2) = \frac{2c_m(h-\beta)u_2}{c_m^2+u_2^2} + \beta, \tag{6.5d}$$

$$\beta = \frac{1+c_m^2-2hc_m}{(1-c_m)^2}. \tag{6.5e}$$

- Inhibitor:

$$g(u_1) = u_1, \tag{6.5f}$$

$$s_c(u_2) = \frac{(h-1)u_2+h}{2(h-1)u_2+1}. \tag{6.5g}$$

α, c_m and h are constants set in the main program (discussed next). Equations (6.5) are first order in t and second order in r; therefore, each PDE requires one IC and two BCs. These auxiliary conditions will be taken as

$$u_1(r,t=0) = 0; u_2(r,t=0) = 0, \tag{6.6a,b}$$

$$u_1(r=1,t) = 1; u_2(r=1,t) = 1, \tag{6.7a,b}$$

$$\frac{\partial u_1(r=0,t)}{\partial r} = 0; \frac{\partial u_2(r=0,t)}{\partial r} = 0. \tag{6.7c,d}$$

Boundary conditions (6.7a) and (6.7b) indicate that the system responds to a unit step in $u_1(r=1,t), u_2(r=1,t)$, starting from the zero IC of eqs. (6.6). Boundary conditions (6.7c,d) can be considered as symmetry conditions at the center of the wound (the BCs for eqs. (6.5) at $r=0$ were not given in [1, 3] so eqs. (6.7c,d), are inferred from the plotted solutions in these references).

6.2.1 Main program

A main program, `pde_2_main`, for eqs. (6.5) to (6.7) follows in Listing 6.4.

```
%
% Wound healing
%
% Clear previous files
  clear all
  clc
%
% Global area
  global      h     cm  alpha   beta...
              u10   Du1     la       ...
              u20   Du2     li       ...
               nr     r  ncase  ncall
%
% Case
%
%   ncase = 1: Activator
%
%   ncase = 2: Inhibitor
  ncase=1;
```

```
%
% Model parameters
  la=30; li=5; h=10; cm=40; alpha=0.1;
  beta=(1+cm^2-2*h*cm)/(1-cm)^2;
  if(ncase==1) Du1=5.0e-04; Du2=0.45; end
  if(ncase==2) Du1=1.0e-04; Du2=0.85; end
  fprintf('\n ncase = %2d\n',ncase);
  fprintf('\n  la = %5.1f     li = %5.1f      h = %5.1f\n',la,li,h);
  fprintf('\n  cm = %5.1f  alpha = %5.3f   beta = %5.3f\n',cm,...
          alpha,beta);
  fprintf('\n Du1 = %8.3e    Du2 = %8.3e\n',Du1,Du2);
%
% Spatial grid
  nr=101; dr=1/(nr-1);
  for j=1:nr
    r(j)=(j-1)*dr;
  end
  u10=1; u20=1;
%
% Independent variable for ODE integration
  if(ncase==1) tf=25; end
  if(ncase==2) tf=50; end
  tout=[0:tf/20:tf]'; nout=21;
  ncall=0;
%
% Initial condition
  for i=1:nr u0(i)=0; u0(i+nr)=0; end
%
% ODE integration
  reltol=1.0e-04; abstol=1.0e-04;
  options=odeset('RelTol',reltol,'AbsTol',abstol);
  [t,u]=ode15s(@pde_2,tout,u0,options);
%
% Display a heading and t at output
  fprintf('\n  nr = %2d\n',nr);
  for it=1:nout
    for i=1:nr
      u1(it,i)=u(it,i);
      u2(it,i)=u(it,nr+i);
    end
    u1(it,nr)=u10; u2(it,nr)=u20;
    fprintf('\n t = %6.2e',t(it));
%
%   Detailed solution
%   for i=1:nr
%     fprintf('\n i = %3d  %5.3f  %5.3f  %5.3f  ',...
%             i,r(i),u1(it,i),u2(it,i));
%   end
  end
  fprintf('\n\n ncall = %5d\n',ncall);
%
```

```
% Plot the solution as u1(r,t), u2(r,t) vs r with t as a parameter
  figure(1)
  plot(r,u1(2:nout,:),'-');
  xlabel('r'); ylabel('u1(r,t)');
  title('u1(r,t)vs r')
  figure(2)
  plot(r,u2(2:nout,:),'-');
  xlabel('r'); ylabel('u2(r,t)');
  title('u2(r,t) vs r')
```

Listing 6.4 Main program `pde_2_main` for eqs. (6.5) to (6.7)

We can note the following details about this code.

- Previous files are cleared and a global area is defined.

```
%
% Wound healing
%
% Clear previous files
  clear all
  clc
%
% Global area
  global      h     cm  alpha   beta...
            u10    Du1     la       ...
            u20    Du2     li       ...
             nr      r  ncase  ncall
```

- The case is selected.

```
%
% Case
%
%   ncase = 1: Activator
%
%   ncase = 2: Inhibitor
  ncase=1;
```

- The parameters in eqs. (6.5) to (6.7) are defined numerically and displayed.

```
%
% Model parameters
  la=30; li=5; h=10; cm=40; alpha=0.1;
  beta=(1+cm^2-2*h*cm)/(1-cm)^2;
  if(ncase==1) Du1=5.0e-04; Du2=0.45; end
  if(ncase==2) Du1=1.0e-04; Du2=0.85; end
  fprintf('\n ncase = %2d\n',ncase);
  fprintf('\n  la = %5.1f     li = %5.1f      h = %5.1f\n',la,li,h);
  fprintf('\n  cm = %5.1f  alpha = %5.3f   beta = %5.3f\n',cm,...
          alpha,beta);
  fprintf('\n Du1 = %8.3e    Du2 = %8.3e\n',Du1,Du2);
```

An explanation of these numerical values follows. `pde_2_main` executes two cases: `ncase=1` for activation and `ncase=2` for inhibition. Equations (6.5) to (6.7) are stated in terms of dimensionless variables and parameters, e.g., $r, t, u_1(r,t), u_2(r,t)$ are dimensionless.

- `la`: λ in eq. (6.5b) for `ncase=1`
- `li`: λ in eq. (6.5b) for `ncase=2`
- `h`: h in eqs. (6.5d,e,g)
- `cm`: c_m in eqs. (6.5d,e)
- `alpha`: α in eq. (6.5c)
- `beta`: β in eq. (6.5d)
- `Du1`: D_{u_1} in eq. (6.5a) for `ncase=1,2`
- `Du2`: D_{u_2} in eq. (6.5b) for `ncase=1,2`

- A spatial grid in r is defined for 101 points (counting the two end points $r = 0, 1$).

```
%
% Spatial grid
  nr=101; dr=1/(nr-1);

  for j=1:nr
    r(j)=(j-1)*dr;
  end
  u10=1; u20=1;
```

The right boundary values (at $r = 1$) in eqs. (6.7a,b) are also defined numerically.

- A time scale in t is defined as $0 \le t \le 25$ with `21` output points for `ncase=1` (counting both end points `t=0,tf`). Similarly, a time scale in t is defined as $0 \le t \le 50$ with `21` output points for `ncase=2`. A longer time scale is required for `ncase=2` to resolve the solution in two plots (discussed subsequently). The longer scale in t was required because of the differences in the parameters for the two cases, particularly `li`, which is smaller than `la` so that the `ncase=2` response is slower.

```
%
% Independent variable for ODE integration
  if(ncase==1) tf=25; end
  if(ncase==2) tf=50; end
  tout=[0:tf/20:tf]'; nout=21;
  ncall=0;
```

The counter for the number of calls to the ODE routine, `pde_2`, is also initialized.

- Initial conditions (6.6) are placed in a single vector `u0` of length $2(101) = 202$. This programming therefore defines the number of ODEs as 202 for the ODE integrator `ode15s` (as the length of `u0`).

```
%
% Initial condition
  for i=1:nr u0(i)=0; u0(i+nr)=0; end
```

- The integration of the $2(nr) = 2(101) = 202$ ODEs is over the interval in t prescribed in `tout`, with the solution returned for the 21 values of t in `tout`. Note that `ode15s` calls the ODE routine, `pde_2`.

```
%
% ODE integration
  reltol=1.0e-04; abstol=1.0e-04;
  options=odeset('RelTol',reltol,'AbsTol',abstol);
  [t,u]=ode15s(@pde_2,tout,u0,options);
```

  The numerical ODE solution is returned in `u`, an array with dimensions `u(21,202)`. t is returned in array `t` of length `t(21)`.
- The solution in `u` is placed in two arrays, `u1,u2`. The boundary values `u1(it,nr)`, `u2(it,nr)` (from BCs (6.7a,b)) are set since these values are not returned by `ode15s` (they are not computed as the solutions to two ODEs at $r = 1$ from the programming in the ODE routine `pde_2`).

```
%
% Display a heading and t at output
  fprintf('\n  nr = %2d\n',nr);
  for it=1:nout
    for i=1:nr
      u1(it,i)=u(it,i);
      u2(it,i)=u(it,nr+i);
    end
    u1(it,nr)=u10; u2(it,nr)=u20;
    fprintf('\n t = %6.2e',t(it));
%
%   Detailed solution
%   for i=1:nr
%     fprintf('\n i = %3d  %5.3f  %5.3f  %5.3f  ',...
%             i,r(i),u1(it,i),u2(it,i));
%   end
  end
  fprintf('\n\n ncall = %5d\n',ncall);
```

  Numerical output is restricted to just the 21 values of t (to give an indication of the progress of the solution). If a more detailed solution is required (at all values of r), the concluding `for` loop can be activated (uncommented). The total number of calls to the ODE routine `pde_2` is also displayed as a measure of the computational effort required to compute the solution.
- The solution is also plotted as u_1, u_2 in two figures.

```
%
% Plot the solution as u1(r,t), u2(r,t) vs r with t as a parameter
  figure(1)
  plot(r,u1(2:nout,:),'-');
  xlabel('r'); ylabel('u1(r,t)');
  title('u1(r,t)vs r')
  figure(2)
  plot(r,u2(2:nout,:),'-');
```

```
    xlabel('r'); ylabel('u2(r,t)');
    title('u2(r,t) vs r')
```

6.2.2 ODE routine

The ODE routine, pde_2, is listed next.

```
  function ut=pde_2(t,u)
%
% Global area
  global      h     cm  alpha   beta...
            u10    Du1     la       ...
            u20    Du2     li       ...
             nr      r  ncase  ncall
%
% One vector to two vectors
  for i=1:nr
    u1(i)=u(i);
    u2(i)=u(i+nr);
  end
%
% BCs at r = 1
  u1(nr)=u10; u2(nr)=u20; nu=1;
%
% ur
  u1r=dss004(0,1,nr,u1);
  u2r=dss004(0,1,nr,u2);
%
% BCs ar r = 0
  u1r(1)=0; u2r(1)=0; nl=2;
%
% urr
  u1rr=dss044(0,1,nr,u1,u1r,nl,nu);
  u2rr=dss044(0,1,nr,u2,u2r,nl,nu);
%
% ut
  for i=1:nr-1
    if(ncase==1) lam=la; sc=sa(u2(i)); g=ga(u1(i)); end
    if(ncase==2) lam=li; sc=si(u2(i)); g=gi(u1(i)); end
    if(i==1)
      u1t(i)=Du1*2*u1rr(i)+sc*u1(i)*(2-u1(i))-u1(i);
      u2t(i)=Du2*2*u2rr(i)+lam*g-lam*u2(i);
    else
    u1t(i)=Du1*(u1rr(i)+(1/r(i))*u1r(i))+sc*u1(i)*(2-u1(i))-u1(i);
    u2t(i)=Du2*(u2rr(i)+(1/r(i))*u2r(i))+lam*g-lam*u2(i);
    end
  end
  u1t(nr)=0; u2t(nr)=0;
%
% Two vectors to one vector
  for i=1:nr
```

```
    ut(i)   =u1t(i);
    ut(i+nr)=u2t(i);
  end

%
% Transpose and counter
  ut=ut';
  ncall=ncall+1;
```

Listing 6.5 ODE routine `pde_2` for eqs. (6.5) to (6.7)

We can note the following details about `pde_2`.

- The function and a global area are defined.

```
  function ut=pde_2(t,u)
%
% Global area
  global      h     cm  alpha   beta...
            u10    Du1     la       ...
            u20    Du2     li       ...
             nr      r  ncase  ncall
```

- The single vector `u` of length 202 that is an input (RHS) argument of `pde_2` is placed in two vectors `u1,u2` to facilitate the programming of eqs. (6.5) to (6.7).

```
%
% One vector to two vectors
  for i=1:nr
    u1(i)=u(i);
    u2(i)=u(i+nr);
  end
```

- Boundary conditions (5.7a,b) are programmed as Dirichlet (`nu=1`) at $r = 1$ (point `nr`).

```
%
% BCs at r = 1
  u1(nr)=u10; u2(nr)=u20; nu=1;
```

- The first derivatives $(\partial u_1/\partial r), (\partial u_2/\partial r)$ are computed by `dss004` (`u1r,u2r`).

```
%
% ur
  u1r=dss004(0,1,nr,u1);
  u2r=dss004(0,1,nr,u2);
```

Note that in calling `dss044`, $0 \le r \le 1$.

- The first derivatives are reset as Neumann (`nl=2`) at $r = 0$ according to BCs (6.7c,d).

```
%
% BCs ar r = 0
  u1r(1)=0; u2r(1)=0; nl=2;
```

- The second derivatives $(\partial^2 u_1/\partial r^2), (\partial^2 u_2/\partial r^2)$ are computed by `dss044` (`u1rr,u2rr`).

```
%
% urr
  u1rr=dss044(0,1,nr,u1,u1r,nl,nu);
  u2rr=dss044(0,1,nr,u2,u2r,nl,nu);
```

- The first derivatives $(\partial u_1/\partial t), (\partial u_2/\partial t)$ are computed according to eqs. (6.5a,b).

```
%
% ut
  for i=1:nr-1
    if(ncase==1) lam=la; sc=sa(u2(i)); g=ga(u1(i)); end
    if(ncase==2) lam=li; sc=si(u2(i)); g=gi(u1(i)); end
    if(i==1)
      u1t(i)=Du1*2*u1rr(i)+sc*u1(i)*(2-u1(i))-u1(i);
      u2t(i)=Du2*2*u2rr(i)+lam*g-lam*u2(i);
    else
    u1t(i)=Du1*(u1rr(i)+(1/r(i))*u1r(i))+sc*u1(i)*(2-u1(i))-u1(i);
    u2t(i)=Du2*(u2rr(i)+(1/r(i))*u2r(i))+lam*g-lam*u2(i);
    end
  end
  u1t(nr)=0; u2t(nr)=0;
```

  The programming of the derivatives in t is done in several steps.

  – At each point in r (each `i`), λ, s_c, g in eqs. (6.5a,b) are computed for `ncase=1` (activation) or `ncase=2` (inhibition).

```
%
% ut
  for i=1:nr-1
    if(ncase==1) lam=la; sc=sa(u2(i)); g=ga(u1(i)); end
    if(ncase==2) lam=li; sc=si(u2(i)); g=gi(u1(i)); end
```

    `ga,sa` are functions for g, s_c for `ncase=1` from eqs. (6.5c,d) discussed subsequently. `gi,si` are functions for g, s_c for `ncase=2` from eqs. (6.5f,g).

  – At $r = 0$ (`i=1`), the first derivative radial groups in r of eqs. (6.5a,b) are indeterminate. They are modified according to l'Hospital's rule.

$$\lim_{r\to 0} \frac{1}{r}\frac{\partial u_1}{\partial r} = \frac{\partial^2 u_1}{\partial r^2},$$

$$\lim_{r\to 0} \frac{1}{r}\frac{\partial u_2}{\partial r} = \frac{\partial^2 u_2}{\partial r^2}.$$

  Thus, the radial groups in eqs. (6.5a,b) become

$$\frac{\partial^2 u_1}{\partial r^2} + \frac{1}{r}\frac{\partial u_1}{\partial r} = 2\frac{\partial^2 u_1}{\partial r^2}, \tag{6.7e}$$

$$\frac{\partial^2 u_2}{\partial r^2} + \frac{1}{r}\frac{\partial u_2}{\partial r} = 2\frac{\partial^2 u_2}{\partial r^2}. \tag{6.7f}$$

  Equations (6.7e,f) are programmed (at $r = 0$ or `i=1`) as

```
if(i==1)
  u1t(i)=Du1*2*u1rr(i)+sc*u1(i)*(2-u1(i))-u1(i);
  u2t(i)=Du2*2*u2rr(i)+lam*g-lam*u2(i);
```

– Equations (6.5a,b) for $r \neq 0$ are programmed as

```
else
u1t(i)=Du1*(u1rr(i)+(1/r(i))*u1r(i))+sc*u1(i)*(2-u1(i))-u1(i);
u2t(i)=Du2*(u2rr(i)+(1/r(i))*u2r(i))+lam*g-lam*u2(i);
end
end
u1t(nr)=0; u2t(nr)=0;
```

The derivatives $(\partial u_1(r=1,t)/\partial t), (\partial u_1(r=1,t)/\partial t)$ are set to zero in accordance with BCs (6.7c,d).

- All 202 derivatives in t are now programmed (the LHS of eqs. (6.5a,b)), so they can be placed in a single vector `ut` to be returned to `ode15s` as a LHS argument of `pde_2`.

```
%
% Two vectors to one vector
  for i=1:nr
    ut(i)   =u1t(i);
    ut(i+nr)=u2t(i);
  end
```

- A transpose of `ut` required by `ode15s` and an increment in the counter `ncall` concludes `pde_2`.

```
%
% Transpose and counter
  ut=ut';
  ncall=ncall+1;
```

The four functions for g, s_c called in `pde_2` are the remaining programming (Listings 6.5a to 6.5d).

```
function ga=gf(u1)
global alpha
ga=u1*(1+alpha^2)/(u1^2+alpha^2);
```

Listing 6.5a Function `ga` from eq. (6.5c)

The programming in `ga` follows directly from eq. (6.5c). Note that a file with the name `ga.m` is required to use (call) this function (the name of the function in the first line, `gf`, is not used).

```
function sa=sf(u2)
global      h     cm   beta
sa=2*cm*(h-beta)*u2/(cm^2+u2^2)+beta;
```

Listing 6.5b Function `sa` from eq. (6.5d)

Again, a file named `sa.m` is required to use (call) this function. `beta` is a constant computed once in `pde_2_main` according to eq. (6.5e) before `sa` is called from `pde_2`.

```
function gi=gf(u1)
gi=u1;
```

Listing 6.5c Function `gi` from eq. (6.5f)

```
function si=sf(u2)
global h
si=((h-1)*u2+h)/(2*(h-1)*u2+1);
```

Listing 6.5d Function `si` from eq. (6.5g)

This completes the programming of eqs. (6.5) to (6.7). The output from the main program `pde_2_main` is considered next.

6.2.3 Two-PDE model output

The output from `pde_2_main` is in two sections for `ncase=1,2`.

ncase = 1

The numerical and graphical output is in Table 6.7 and Figs 6.5 and 6.6.

We note that to compute a complete solution, $0 \le t \le 25$, `14483` calls to `pde_2` were required. This is not a prohibitively large number and the solution was computed in a few minutes on a modest desktop computer.

The plotted solutions are given in Fig. 6.5 ($u_1(r,t)$) and Fig. 6.6 ($u_2(r,t)$).

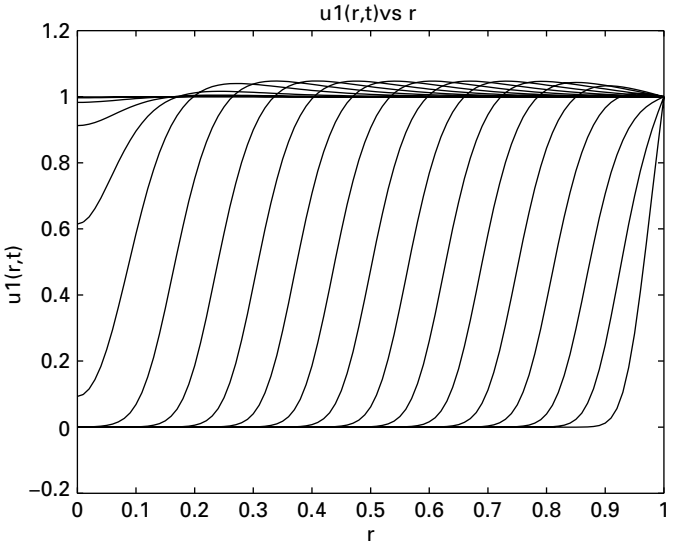


Figure 6.5 $u_1(r,t)$ from eqs. (6.5) to (6.7) for `ncase=1`

Table 6.7. Numerical output from `pde_2_main` of Listing 6.4 for `ncase=1`

```
 la =  30.0     li =   5.0      h =  10.0

 cm =  40.0  alpha = 0.100   beta = 0.527

Du1 = 5.000e-004    Du2 = 4.500e-001

 nr = 101

t = 0.00e+000
t = 1.25e+000
t = 2.50e+000
t = 3.75e+000
t = 5.00e+000
t = 6.25e+000
t = 7.50e+000
t = 8.75e+000
t = 1.00e+001
t = 1.13e+001
t = 1.25e+001
t = 1.38e+001
t = 1.50e+001
t = 1.63e+001
t = 1.75e+001
t = 1.88e+001
t = 2.00e+001
t = 2.13e+001
t = 2.25e+001
t = 2.38e+001
t = 2.50e+001

ncall = 14483
```

The moving front characteristic of $u_1(r,t)$ as the wound closes is clear. Also, these solutions appear to essentially reproduce the solutions in [1], although a direct comparison is not possible since numerical output from [1] is not given, and the plotted output does not include the parametric values of t.

ncase = 2

The numerical and graphical output is in Table 6.8 and Figs 6.7 and 6.8.

We note that to compute a complete solution, $0 \leq t \leq 50$, `1287` calls to `pde_2` were required, less than $1/10$ the number for `ncase=1`. This large reduction in the calls can be attributed simply to parameter changes between `ncase=1` and `ncase=2` since all of the code is the same for both cases. In other words, the change in the value of `ncall`

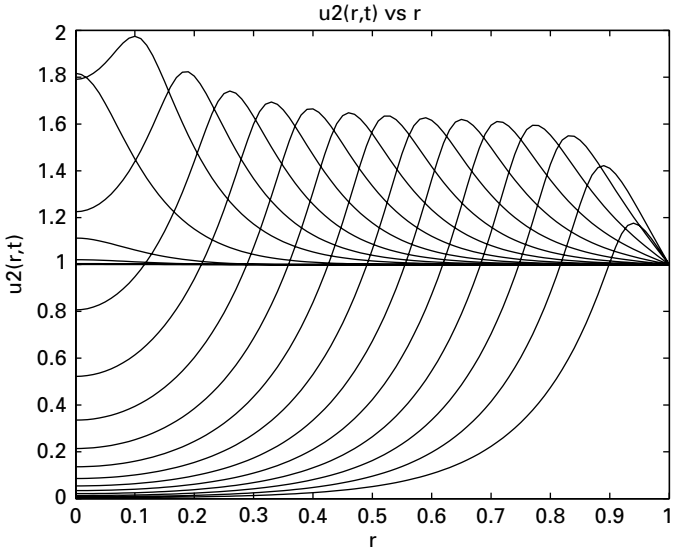


Figure 6.6 $u_2(r,t)$ from eqs. (6.5) to (6.7) for `ncase=1`

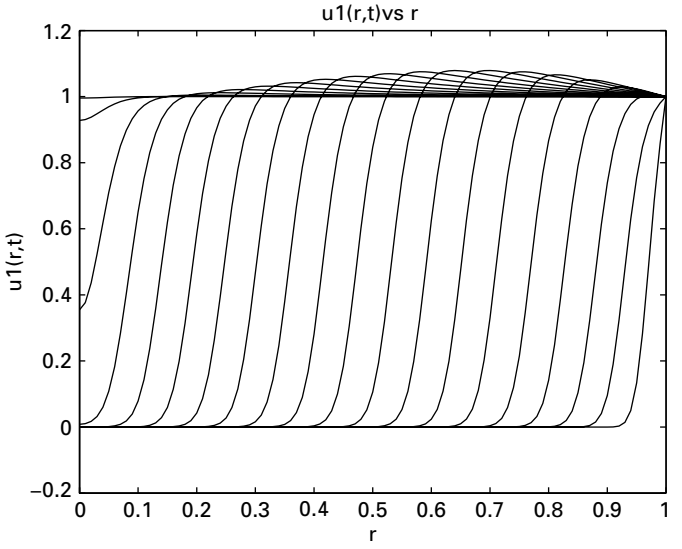


Figure 6.7 $u_1(r,t)$ from eqs. (6.5) to (6.7) for `ncase=2`

clearly indicates the sensitivity of the numerical calculations to the particular parameter values.

The plotted solutions are given in Figs. 6.7 ($u_1(r,t)$) and Figs. 6.8 ($u_2(r,t)$).

Again, the moving front characteristic of $u_1(r,t)$ as the wound closes is clear. The velocity is lower for `ncase=2`, thereby necessitating a longer time scale (to $t = 50$ for `ncase=2` rather than $t = 25$ for `ncase=1`). As before for `ncase=1`, these solutions appear

Table 6.8. Numerical output from `pde_2_main` of Listing 6.4 for `ncase=2`

```
 la =  30.0     li =   5.0      h =  10.0

 cm =  40.0  alpha = 0.100   beta = 0.527

Du1 = 1.000e-004    Du2 = 8.500e-001

 nr = 101

t = 0.00e+000
t = 2.50e+000
t = 5.00e+000
t = 7.50e+000
t = 1.00e+001
t = 1.25e+001
t = 1.50e+001
t = 1.75e+001
t = 2.00e+001
t = 2.25e+001
t = 2.50e+001
t = 2.75e+001
t = 3.00e+001
t = 3.25e+001
t = 3.50e+001
t = 3.75e+001
t = 4.00e+001
t = 4.25e+001
t = 4.50e+001
t = 4.75e+001
t = 5.00e+001

ncall =  1287
```

to essentially reproduce the solutions in [1], although a direct comparison is not possible since numerical output from [1] is not given, and the plotted output does not include the parametric values of t.

This completes the discussion of the two-PDE model. The main features of this model are two simultaneous (coupled), nonlinear PDEs, which would most likely be beyond analytical methods, but which are straightforward with a numerical approach. We conclude this chapter with a few additional observations.

6.3 Conclusions concerning space and time discretizations

The preceding discussion indicates that the MOL programming of the PDE models required two basic decisions: (1) the selection of a discretization in space, and (2) the

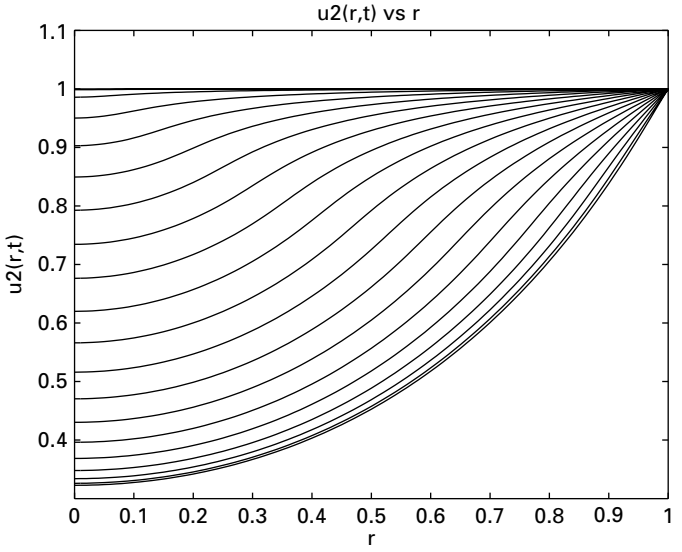


Figure 6.8 $u_2(r,t)$ from eqs. (6.5) to (6.7) for `ncase=2`

selection of an ODE integrator; the latter is also a form of discretization (the ODE integrator takes discrete steps in t). The process of selecting these two discretizations is generally by experimentation, with no guarantee of success in advance. In the present case, the spatial discretization was by `dss004` and `dss044` ($O(h^4)$ where h is the spatial grid spacing), which are our usual choice, at least to start. Then some experimentation with the order of the spatial discretization is easily carried out just by calling two alternative routines, e.g., `dss006` and `dss046`, which are $O(h^6)$. This form of p refinement is recommended, although unexpectedly, using higher order FD approximations may cause the calculation to fail (since these higher order FD approximations are based on higher order polynomials that can oscillate more than lower order polynomials and therefore cause the calculations to fail).

Another test would be to change the spatial grid spacing by varying the number of grid points (`nr = 101` in the present case). Since for `ncase=1`, the number of calls to `pde_2` was relatively large (`14483`), we decided to reduce `nr` to reduce the computational effort. Therefore `nr` was reduced from `101` to `51`. This had two unexpected effects:

- The plotted profiles in r had a nonsmooth appearance, indicating the grid was too coarse.
- The ODE integration by `ode15s` failed for `ncase=2`. Specifically, the integration stopped with an error message before the final value of t was reached, to warn that indicated the specified error tolerances (`abserr, relerr` set in `pde_2_main` before the call to `ode15s`) could not be achieved.

Clearly, neither of these outcomes is acceptable; they are presented as an example of what might go wrong with h (and p) refinement.

The t discretization can also be done in two ways. The order of the ODE integrator can be changed, such as changing from the modified Euler method ($O(h^2)$) to the Runge Kutta integrator ($O(h^4)$), as discussed in Chapter 1. p refinement is actually done automatically and internally within `ode15s` as the integration proceeds from order 1 to order 5 (which is the reason for the `15` designation in the name `ode15s`). h refinement is also carried out automatically and internally by `ode15s` in attempting to meet the specified error tolerances.

`ode15s` is a high quality integrator, and is therefore the basis for much of our MOL analysis, especially when the sparse option is included. However, it can fail, as when `nr` was changed from `101` to `51` for `ncase=2`, typically for no apparent reason, and therefore with no apparent remedy. Since `ode15s` is a large routine with hundreds of lines of code, trying to determine what went wrong is nearly impossible (except by the authors who wrote it). When this happens, we often next go to a simpler integrator, even a fixed-step Euler integrator. This has the advantage of our knowing in detail what it is doing, possibly with printing intermediate output to try to identify the underlying problem. But there is no guarantee that this will find the source of the computational problem, and additional experimentation and investigation might be required, such as taking just one small step away from the initial condition.

In summary, the development of a new MOL code usually requires some experimentation. This was true for the PDE models and codes discussed in this book. Then once some numerical output is produced, a critical review, usually with an error analysis such as h and p refinement, is essential. On the other hand, the success rate for developing MOL solutions to complex PDE problems (which are usually beyond analytical methods) is quite high.

References

[1] Murray, J. D. (2003), *Mathematical Biology, II: Spatial Models and Biomedical Applications*, 3rd Edn, 444–454

[2] Schiesser, W. E. and Griffiths, G. W, (2009), *A Compendium of Partial Differential Equation Models*, Cambridge, UK: Cambridge University Press

[3] Sherratt, J. A. and Murray, J. D. (1990), Models of epidermal wound healing, *Proc. R. Soc. Lond. B*, **241** (1300), 29–36